TeSpec: Proof-Based Testing of Separation-Logic Specifications for C

Anonymous Authors

***Abstract*—Fixing a program's input does not necessarily make its logical specification executable. A finite execution can leave quantified properties and spatial ownership conditions to establish. We present TESPEC, a method for proof-based testing of separation-logic specifications for C. The method uses execution to determine program states and proof to check the specification at those states. Symbolic execution follows function bodies and loop iterations under selected input bindings. Annotations generate obligations on copies of the reached states, allowing execution to continue with the values and memory it produced. Automatic reasoning and proof-assistant completion check these obligations in the original specification logic. We formalize instance checking and mechanize trace-preservation and acceptance arguments for a reference model. Recorded case studies examine a signed-overflow violation missed by a Fulminate configuration, original quantified contracts with equivalent runtime controls, and spatial obligations resolved by automatic reasoning or checked refutation. TESPEC supports execution-based feedback from proof-oriented contracts before full program verification is complete.**

***Index Terms*—Specification testing, symbolic execution, separation logic, proof checking**

I. INTRODUCTION

Fixing a program's input does not necessarily make its specification executable. Consider a routine that returns the least nonnegative integer whose square is at least 13. The result is 4, but its contract may characterize leastness by relating that result to *every* nonnegative integer. The computation is finite; the resulting assertion still quantifies over an unbounded domain. For stateful C programs, the contract must also describe the memory that the routine reads, modifies, and owns.

Separation logic expresses these functional and spatial requirements in a common language [1], as used by CN and VeriFast [2], [3]. Fulminate brings executable CN contracts to runtime testing [4]. Proof-oriented specifications also contain mathematical relations whose computational interpretation is less direct. Some admit a finite monitor, but discovering that monitor and establishing its equivalence requires additional specification work. A programmer can therefore write a precise contract and run a concrete example, yet still lack an oracle for checking it against the contract as written [5].

We present TESPEC, a method for *proof-based testing of separation-logic specifications for C*. Its central idea is to use execution to determine program states and proof to check the specification at those states. Input bindings restrict the initial domain. Symbolic execution follows available function bodies and loop iterations, retaining the values and memory resources they produce. Selected annotations become propositions in the original verification logic. Automatic reasoning discharges supported obligations; remaining propositions become proof-assistant tasks.

Realizing this idea requires separating an observation from the execution it observes. A verifier may replace a loop by its invariant or a function body by its summary [6]. When that annotation is itself under test, replacement can discard the information needed to check it. TESPEC instead generates obligations on copies of the reached state. For a state assertion P and a fully resolved check Q_A, it records $P \vdash Q_A$ while retaining P for the continuation. Changing an intermediate assertion changes the question asked of the state; subsequent computation continues from the same state. Section II illustrates this mechanism with the least-root routine.

The construction preserves the relation between heap cells and their live values during simplification and accounts explicitly for local storage at observation boundaries. The oracle combines VC evidence with entry feasibility and execution coverage. Acceptance concerns the domain selected by the bindings, which may retain symbolic addresses or other unbound inputs. Full QCP verification addresses the original contract over all admissible inputs. These workflows share specifications and proof infrastructure while answering different development questions.

Recorded case studies examine the mechanism from three directions. For a negating read at `INT_MIN`, TESPEC rejects an arithmetic safety obligation while the recorded Fulminate configuration exits successfully. For least-root and divisibility, proof completion checks original quantified contracts; proved finite reformulations provide runtime controls. Ownership examples examine how automatic spatial reasoning and checked refutations resolve residual obligations. Section V presents their sources, inputs, and evidence.

Our contributions are:

1) **Execution with copied observations.** We construct obligations at function, loop, and intermediate boundaries while retaining the program's evolving state (Section III).
2) **A formal account of instance checking.** We give construction rules, oracle interfaces, and verdict judgments, with trace-preservation and acceptance arguments supported by a mechanized reference model (Section IV).
3) **An implementation and comparative evidence.** We implement the method using QCP's symbolic execution and Rocq obligations, provide controlled case studies, and specify the broader evaluation and cost protocol (Sections V and VI).

II. A Running Example

Consider a routine that replaces an owned integer by its least nonnegative integer square root rounded upward. The contract relates the initial value x, the returned value, and the final cell contents. Its imported Rocq proposition is

$$\mathsf{LeastRoot}(n, r) \equiv 0 \le r \wedge \forall k \in \mathbb{Z}.\, 0 \le k \Rightarrow (r \le k \Leftrightarrow n \le k^2). \tag{1}$$

Listing 1 shows the C body and abbreviated annotations of the archived instance. We use $p \mapsto_{\text{int}} v$ for typed cell ownership; the artifact uses QCP's `data_at` notation.

```
int root_cell(int *p)
/*@ With x
    Require 0 <= x && x <= 100 &&
            data_at(p, int, x)
    Ensure least_root(x, __return) &&
           data_at(p, int, __return) */
{
  int n = *p;
  int r = 0;
  /*@ Inv n == x && 0 <= r && r <= 10 &&
      (r == 0 || (r-1)*(r-1) < n) &&
      data_at(p, int, x) */
  while (r*r < n) { r = r + 1; }
  /*@ Assert least_root(n, r) &&
              data_at(p, int, x) */
  *p = r;
  return r;
}
```

Listing 1. Least-root instance with an intermediate observation. The imported relation is Equation (1).

a) A logical instance.: The binding $\theta = \{x \mapsto 13\}$ fixes the logical input and retains the address p. The spatial precondition describes the cell at that symbolic address. Loop execution produces $r = 0, 1, 2, 3, 4$ at its five invariant boundaries; then the body writes 4 and returns 4. The final observation is

$$\mathsf{LeastRoot}(13, 4) \wedge p \mapsto_{\text{int}} 4.$$

Local scalar storage is handled by the observation frame described in Section III-B. The imported relation remains a proposition in the verification logic. Automatic execution establishes the entry and safety phases, and proof completion checks the remaining functional and observation obligations against their original generated statements.

b) Testing an intermediate specification.: Replacing the assertion by $\mathsf{LeastRoot}(n, r-1)$ changes the observation while leaving the C body and final contract intact. At that boundary the program still has $n = 13, r = 4$, so the candidate asks for $\mathsf{LeastRoot}(13, 3)$. Instantiating its universal quantifier with $k = 3$ exposes the contradiction $13 \le 9$. The failed observation does not replace the reached state or alter the following write. Both variants therefore follow the same program computation.

c) Original assertions and executable reformulations.: The original relation ranges over all nonnegative integers even after n and r are fixed. An executable reformulation is

$$0 \le r \wedge n \le r^2 \wedge \big(r = 0 \vee (r-1)^2 < n\big). \tag{2}$$

The two formulas are equivalent over mathematical integers; the C contract bounds inputs to control machine arithmetic. Constructing and justifying the finite form is one route to a runtime oracle. TESPEC instead retains the original relation as a proposition and reuses its proof library. The checked comparison appears in Section V.

III. Checking Stateful Instances

Figure 1 follows a test through three stages: select an instance, execute its bodies while recording observations, and discharge the resulting propositions. The execution supplies states; the oracle supplies evidence about those states.

A. Select an instance and execute its bodies

Typed bindings specialize the root precondition. Binding x to 13 can leave the address p symbolic. The entry checker establishes that the selected domain contains a state, using automatic reasoning or an existential Rocq obligation. Execution retains the entire domain. An existence witness certifies nonemptiness without selecting the sole heap to execute.

TESPEC reuses QCP [7] to evaluate expressions, update symbolic heaps, and generate arithmetic and memory-safety obligations. Known guards select a successor; unresolved guards retain each successor whose infeasibility has not been established. Calls execute available bodies with fresh parameter slots and logical entry bindings. Shared writes and the actual return value survive the call; callee-local slots are retired. Loops execute their conditions and bodies, including `break`, `continue`, and early return. An exhausted resource bound records an incomplete execution.

For example, a helper that increments a cell from 3 to 4 leaves the caller with 4 even if its contract promises only some integer greater than 3. The contract is checked against the computed state. The concrete value remains available to subsequent operations.

B. Observe states without changing them

At a selected boundary, let P describe the reached state and A the resolved annotation. The checker records

$$P \vdash A * F_{P,A}$$

on a logical copy and continues with P. Figure 2 shows the consequence: changing the least-root assertion changes its VC while the same write and return still follow. The copy is a logical snapshot, not a runtime duplication of the heap.

a) Resolve the observed footprint.: Scalar C locals occupy owned cells. An auxiliary annotation may omit unrelated local slots, so $F_{P,A}$ contains exactly those eligible slots, with their types and current values. Resolution exposes local addresses before framing, including a local pointer that aliases another local, to avoid counting a cell twice. User-owned cells omitted by an assertion remain observable obligations. For a mask m selecting local cells, exact heap decomposition gives

$$(A * \mathit{exact}(h_m))(h) \Longleftrightarrow A(h_{\neg m}).$$

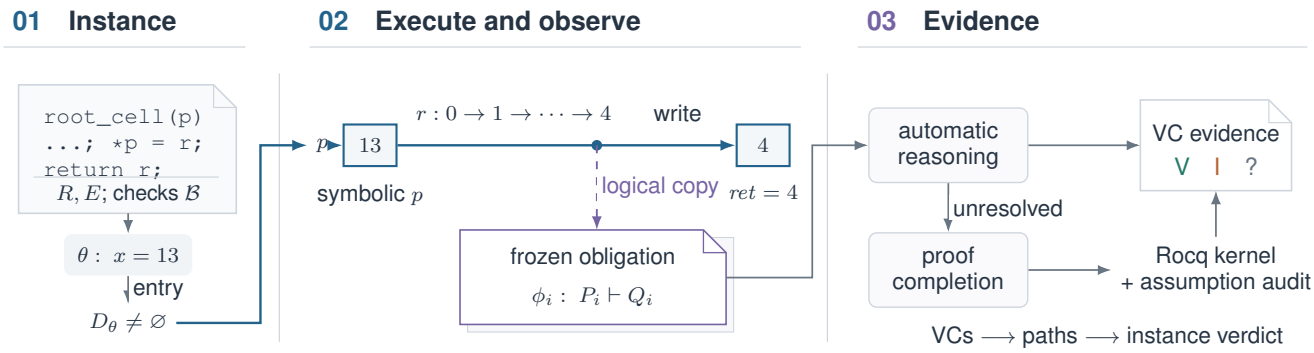


Fig. 1. Instance checking in TESPEC. Binding $x = 13$ retains a symbolic address p; execution updates its cell from 13 to 4. A dashed copy produces an obligation without changing the execution. Automatic decisions and kernel-checked completion populate the VC record (V: VALID, I: INVALID, ?: UNKNOWN); the automatic route has its own soundness interface (Section IV).

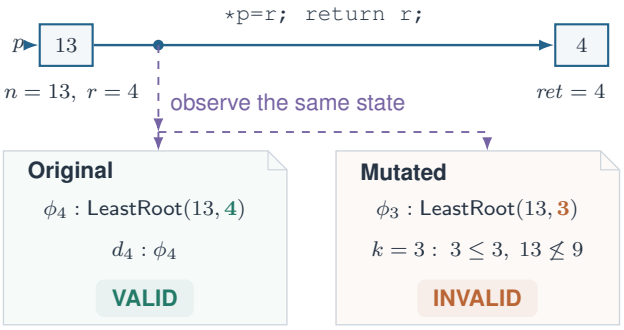


Fig. 2. One reached state, two assertions. Matching the shared spatial resources leaves the displayed functional goals. A proof d_4 establishes the original goal; $k = 3$ refutes the mutation's biconditional. Both annotations retain the same write and return. VALID and INVALID classify VCs; path failure also requires reachability.

The mask may depend on an existential witness. Resolution retains the witness choices; refuting one choice does not refute all of them.

b) Record the selected boundaries.: The schedule identifies function, loop, and intermediate observations. Both supported execution policies run bodies and loops; the richer policy also checks selected internal annotations. Named contract alternatives are resolved by explicit routing or retained as a disjunction. An unresolved boundary match records incomplete coverage. The formal observation inventory in Section IV connects this schedule to the checks actually emitted.

C. Joint heap and value normalization

Symbolic execution retains values outside its current heap formula: return terms, saved call arguments, entry values, and pending checks. Normalization must preserve their relationship to the heap. Projecting away r from $r = 2 \wedge \mathit{cell}(p, 1)$ can preserve the existentially closed heap assertion while losing the value of a return term that still refers to r. We therefore normalize an assertion P together with its live terms $\bar{t}$. Split the logical environment into immutable root bindings ρ and auxiliary witnesses ξ. Their joint denotation is

$$\mathrm{Obs}_\rho(P, \bar{t})(s, \bar{v}) = \exists \xi.\ P(s, \rho, \xi) \wedge \bar{v} = \mathrm{eval}(\bar{t})_{s,\rho,\xi}.$$

An admitted reduction from $(P, \bar{t})$ to $(P', \bar{t}')$ preserves this relation pointwise for each ρ. Live terms include every value used by the continuation or a selected check; these uses must depend only on the preserved state, root bindings, and live values.

```
Normalize(Gamma, P, liveTerms, budget):
  while budget > 0 and a reduction is admitted:
    resolve the rule in Gamma
    establish its guard from P
    freshen typed witnesses
    replace the complete equivalent expression
    update dependent liveTerms consistently
    simplify exposed computations
    budget := budget - 1
  return (P, liveTerms)
```

Listing 2. Normalize the heap and its live values together.

A guarded rule requires an equation in the selected interpretation,

$$\Gamma \models g(\bar{a}) \Rightarrow \big(H(\bar{a}) \leftrightarrow \exists \bar{w}.\, B(\bar{a}, \bar{w})\big),$$

and validity of $g(\bar{a})$ in the reached state. An unresolved guard retains the original application. Unfolding preserves every pure constraint, resource, alternative, and typed witness. A checked pure consequence may instead be exposed by $H \leftrightarrow H \wedge G$, retaining the spatial predicate.

Fresh witnesses cannot capture live variables. Even a syntactically unused witness of nominal type T carries an existence condition:

$$(\exists w : T.\, P) \leftrightarrow (\mathit{inhabited}(T) \wedge P), \qquad w \notin \mathsf{FV}(P).$$

Integer and list types have canonical witnesses; arbitrary nominal types retain the inhabitance obligation. Shared-base offsets and closed list operations can simplify while addresses remain symbolic. C bounds and arithmetic safety remain required checks.

`DefinitionNormalization.v` proves that composing licensed reductions preserves this joint denotation, nonemptiness, validity, and countermodel queries. Rule licensing is an explicit premise. Budget exhaustion returns the residual expression with its meaning preserved.

D. Discharge and aggregate evidence

Figure 3 separates evidence for a VC from evidence for an execution. Automatic entailment first attempts each frozen original obligation. SMT procedures can combine separation constraints with background theories [8], [9]. Our SMT route searches for a model of the negated obligation; the optional `--spatial-smt` mode encodes supported memory predicates. UNSAT establishes validity under the encoding interface. SAT establishes invalidity after checking the model in the original interpretation. An unconfirmed model or unresolved query enters proof completion.

The agent or developer receives the original proposition ϕ and can construct a proof of ϕ or $\neg\phi$. Rocq checks the candidate, and the dependency audit records its assumptions. Separating proof search from acceptance also underlies CoqHammer reconstruction and SMTCoq certificate checking [10], [11]. All residual VCs from one test execution form one completion job, with an individual result for each VC. The manifest fixes source hashes, target interpretation, goal identity, and proof polarity. The optional complete-certificate route also replays automatically discharged goals and entry existence.

The evidence ledger then combines VC status with execution evidence. A reachable violating state establishes path FAIL; a feasible, complete path whose required VCs are all valid establishes PASS. Other paths remain UNKNOWN. At instance level, acceptance additionally requires a nonempty entry and coverage of its executions. An empty entry is reported with an invalid-instance reason. Section IV gives the rules and their correctness arguments.

IV. Instance Semantics and Guarantees

We distinguish constructing an execution tree, establishing its obligations, and assigning a verdict. A failed proof does not change execution, and a refuted abstract obligation does not by itself establish an execution counterexample [12].

A. Semantic objects

A C configuration $\sigma = \langle \ell, \nu, h, \kappa \rangle$ contains its control location, variable environment, owned heap, and call stack. Frames retain their invocation's logical entry values. A semantic state $z = (\sigma, \rho, \bar{v})$ additionally records immutable root bindings ρ and labelled live logical values $\bar{v}$. Auxiliary proof witnesses are not components of z. The supplied transition relation $z \to_c z'$ preserves ρ and updates the remaining components according to execution.

Assertions use the interface syntax

$$A ::= p(\bar{t}) \mid \mathsf{emp} \mid t \mapsto_T u \mid A * A \mid A \wedge A \mid A \vee A \\ \mid \exists x : T.\, A \mid \forall x : T.\, A.$$

A fixed interpretation Γ supplies predicate definitions, typed resources, and layout. Write $A(z)$ for satisfaction. Ownership is exact: $\mathsf{emp}(h)$ means $h = \varnothing$, and $(A * B)(h)$ requires $h = h_1 \uplus h_2$ with disjoint components satisfying A and B [1].

Definition 1 (Specification instance)**.** *An instance is $I = \langle c, R, E, \theta, \mathcal{B}, \Gamma, b \rangle$: a body, entry and return assertions, typed partial binding, observation schedule, interpretation, and resource bounds. Its domain is*

$$D_\theta = \{z = (\sigma, \rho, \bar{v}) \mid \mathsf{Entry}_c(z) \wedge \rho \models \theta \\ \wedge R(z)\}. \tag{3}$$

Entry_c fixes the entry location and initializes the program and live-value environments.

An instance check quantifies over all of D_θ. Establishing its nonemptiness does not replace this domain by the existence witness.

Each source check has an identity α and an independent meaning $\mathsf{Check}_\Gamma(\alpha, z)$: interpret its original annotation in the current invocation, including the exact eligible local-storage frame. Resolution must produce a predicate Q_α satisfying

$$\forall z \in S_N. \quad Q_\alpha(z) \iff \mathsf{Check}_\Gamma(\alpha, z). \tag{4}$$

Here S_N is the represented state set defined below. This interface preserves invocation bindings, all existential witness choices and their frames, and user-owned resources. Acceptance uses the forward direction; refutation uses the reverse direction.

B. Candidate execution and observation rules

A symbolic node $N = \langle \chi; P, \bar{t}, U, \Omega \rangle$ contains its control and invocation context χ, an assertion, live terms, pending check identities, and an append-only obligation inventory. Each record stores its check identity, source state predicate, and target predicate; its proposition is frozen when the record is created. Its represented states use the joint denotation defined in Section III-C:

$$S_N = \{(\sigma, \rho, \bar{v}) \mid \sigma \sim \chi \ \wedge \\ \mathrm{Obs}_\rho(P, \bar{t})(\sigma, \bar{v})\},$$

where $\sigma \sim \chi$ fixes the represented control location and call context. Thus normalization may change auxiliary witnesses while preserving that context, root bindings, and live values.

The following rules construct a candidate tree; construction does not assume its obligations are valid. Program execution uses

$$\frac{U = [] \qquad \mathsf{Transfer}_c(N) = [N_1, \ldots, N_k]}{N \overset{\text{exec}}{\Longrightarrow} \{N_1, \ldots, N_k\}}.$$

Children inherit the obligation inventory and receive the observations selected at their new boundaries. Certification requires safety, progress, and successor coverage:

$$\forall z \in S_N.\, \mathsf{Safe}(z) \wedge \neg \mathsf{Final}(z), \\ \forall z \in S_N.\, \exists z'.\, z \to_c z', \\ \mathsf{Post}_c(S_N) \subseteq \bigcup_i S_{N_i}, \tag{5}$$

where $\mathsf{Post}_c(S) = \{z' \mid \exists z \in S.\, z \to_c z'\}$. Unresolved guards retain each successor not proved infeasible. Coverage permits overapproximation.

At an observation, create the record $e = (\alpha, S_N, Q_\alpha)$, whose proposition is $\phi_e \equiv \forall z \in S_N.\, Q_\alpha(z)$. The rule records e without changing the continuation:

$$\frac{U = \alpha :: U' \qquad Q_\alpha = \mathsf{Resolve}_\Gamma(N, \alpha)}{N \overset{\text{obs}}{\Longrightarrow} N[U \leftarrow U',\ \Omega \leftarrow \Omega \cdot [e]]}. \tag{6}$$

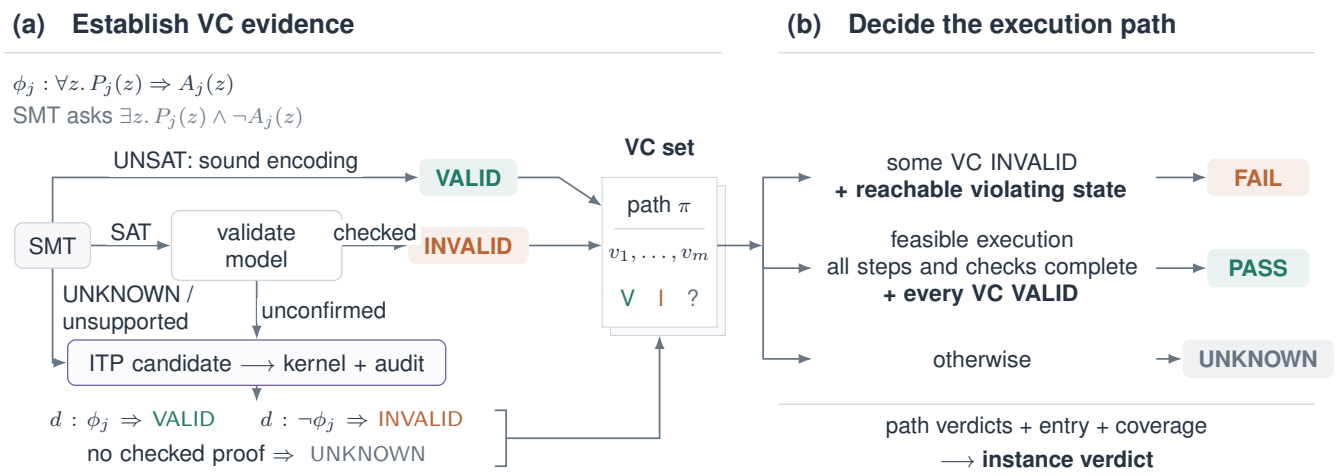


Fig. 3. From solver answers to execution verdicts. (a) SMT checks the negated obligation. A validated countermodel or a checked proof fixes its polarity; unresolved obligations remain UNKNOWN. (b) Path FAIL additionally requires a reachable violation; PASS requires feasible, complete execution and all required checks. Table I gives instance aggregation. Entry has a separate query: SAT establishes nonemptiness, while UNSAT identifies an invalid empty test.

The field update preserves χ, P, and $\bar{t}$. Neither ϕ_e nor its negation is a premise. Resolution faithfulness (Equation (4)) is required when certifying the tree.

Write $N \equiv_J N'$ when only $(P, \bar{t})$ changes and its joint denotation is preserved pointwise for every ρ. Normalization uses

$$\frac{N \equiv_J N'}{N \overset{\text{norm}}{\Longrightarrow} N'}.$$

Therefore $S_N = S_{N'}$, retaining both the heap and its live values.

Administrative progress requires a well-founded rank decreasing on every observation or normalization step between program transitions. This is a reference-construction condition, independent of the implementation's choice of resource bounds. Exhausting a bound creates an UNFINISHED leaf. A DONE node has no pending observations and must establish safety, finality, E, and absence of outgoing program transitions.

Definition 2 (Boundary completeness)**.** *Let* $\mathsf{rec}_{\mathcal{B}}(z)$ *list the selected pairs* (α, z)*;* $\mathsf{Observe}_{\mathcal{B}}(\tau)$ *concatenates them along* τ*. A concrete trace embeds in a tree branch by matching program steps to execution edges and retaining its state across administrative edges. Let* $\Omega_\pi^{\mathrm{obs}}$ *be that branch's ordered observation records, counting each once. The projection* inst_τ *replaces a record* (α, S, Q) *by* (α, z_i) *at its matching observation edge. The tree is boundary-complete when every complete embedded branch satisfies*

$$\mathsf{inst}_\tau(\Omega_\pi^{\mathrm{obs}}) = \mathsf{Observe}_{\mathcal{B}}(\tau).$$

The state at each matched edge belongs to its record's source set. List equality preserves check identity, order, and repeated loop visits. Together with faithful resolution, it connects valid recorded propositions to the original selected annotations.

C. Oracle evidence and verdicts

For an original obligation j, define

$$\phi_j = \forall z.\, S_j(z) \Rightarrow A_j(z),$$
$$C_j = \exists z.\, S_j(z) \wedge \neg A_j(z).$$

C_j is its classical countermodel query. A supported SMT encoding B_j and original-model validator must satisfy

$$\begin{aligned} C_j &\Rightarrow \exists m.\, B_j(m), \\ \mathsf{Validate}_j(m, z) = \mathsf{true} &\Rightarrow S_j(z) \wedge \neg A_j(z). \end{aligned} \tag{7}$$

Proposition 1 (Oracle soundness)**.** *Assuming sound solver answers and Equation* (7)*, UNSAT establishes* ϕ_j*; successful original-model validation establishes* $\neg\phi_j$*. A kernel-checked proof establishes its checked polarity when its statement denotes the original obligation and* Γ *satisfies its admitted dependency theory.*

Proof. If C_j held after UNSAT, the first interface condition would produce a model of B_j, a contradiction. Validation supplies a state satisfying S_j and violating A_j, contradicting ϕ_j. The proof-assistant case follows from kernel soundness under the stated interpretation of its dependencies. □

SAT without validated reconstruction, unsupported encodings, and unsuccessful proof attempts establish neither polarity. In particular, proving $\neg\phi_j$ need not construct a reachable model.

Entry uses the separate proposition $\exists z.\, z \in D_\theta$: SAT means nonempty, and UNSAT means empty. The executor's negative-entailment entry encoding has this meaning under the explicit classical premise in `PreconditionOracle.v`.

For a branch π, let F_π mean actual feasibility from D_θ, K_π mean complete execution and selected checks, and W_π mean that a refuted safety, observation, or return VC has a validated

TABLE I
INSTANCE VERDICTS IN PRIORITY ORDER.

Verdict	Evidence
FAIL: empty	Entry UNSAT: an invalid test instance.
FAIL: violation	Some branch has a reachable violating state.
PASS	Entry SAT, a complete covering tree, and all retained branches PASS.
UNKNOWN	None of the preceding conditions is established.

violating state reached at that check. With original VC results v_j,

$$V(\pi) = \begin{cases} \mathsf{FAIL}, & W_\pi, \\ \mathsf{PASS}, & F_\pi \wedge K_\pi \wedge \forall j.\, v_j = \mathsf{VALID}, \\ \mathsf{UNKNOWN}, & \text{otherwise.} \end{cases}$$

Rules have priority order: a reachable violation suffices despite other unresolved obligations. Vacuous validity on an infeasible branch does not establish a passing execution. Removing infeasible branches must preserve coverage.

Execution errors retain separate diagnostics. Refuting a coverage or normalization side condition prevents acceptance; it is not itself a program violation.

D. Reference guarantees

The monitored judgment $z \Longrightarrow_{\mathcal{B}}^{\tau,\omega} z'$ records a finite trace and its observations:

$$\frac{}{z \Longrightarrow_{\mathcal{B}}^{[z],\, \mathsf{rec}_{\mathcal{B}}(z)} z}.$$

$$\frac{z \to_c u \qquad u \Longrightarrow_{\mathcal{B}}^{\tau,\omega} z'}{z \Longrightarrow_{\mathcal{B}}^{z::\tau,\, \mathsf{rec}_{\mathcal{B}}(z)\cdot\omega} z'}.$$

Theorem 1 (Erasure and lifting)**.** *A state sequence τ admits a monitored derivation from z to z' if and only if it is a finite $\to_c$ trace between those states. Every such derivation has $\omega = \mathsf{Observe}_{\mathcal{B}}(\tau)$.*

Proof. Induction on a program trace constructs a monitored derivation using the singleton and step rules. Conversely, induction on a monitored derivation retains exactly its program-step premises. The same induction establishes the observation-list equation. No rule tests an observation's truth. □

The theorem establishes reference trace preservation. Resource bounds add a separate completeness check to the implementation.

Theorem 2 (Nonvacuous instance acceptance)**.** *Assume sound oracle evidence, a root representing exactly D_θ, and valid execution, normalization, and resolution interfaces. If a finite boundary-complete tree without unfinished leaves receives PASS, then $D_\theta \neq \varnothing$ and every maximal execution from D_θ terminates safely, satisfies E, and satisfies every selected observation.*

Proof. Entry SAT supplies nonemptiness. Induct on tree height for a represented state. Terminal obligations establish the DONE case. At an execution node, progress excludes stuck nonfinal states and coverage places every successor in a child subtree. Normalization preserves the joint state set; observation retains it and establishes the recorded predicate by oracle soundness. Faithful resolution transfers that result to the original annotation. Each program step descends the finite tree, so an execution cannot have more program steps than its height. Every maximal execution therefore terminates. Boundary completeness identifies the discharged inventory with every selected observation on that execution. □

A confirmed violation means

$$\exists z_0 \in D_\theta, z.\ z_0 \to_c^* z \wedge \mathsf{Bad}_{E,\mathcal{B}}(z), \tag{8}$$

where badness is unsafety, failure of a selected observation, or failure of E at return.

Proposition 2 (Reachable-refutation soundness)**.** *Assume the resolution interface. A validated countermodel to a safety, observation, or return VC establishes Equation* (8) *when an admitted trace reaches it at the corresponding check. Such evidence excludes acceptance of the same instance.*

Proof. Validation supplies the violated predicate. Faithful resolution and the matched check identity transfer this violation to the original check, establishing badness. The trace supplies the initial state and reachability. Acceptance would establish safety and the same selected predicate at that state by Theorem 2, a contradiction. □

The reachability condition is essential: an abstract endpoint may admit $x = 1$ and refute a requirement $x = 0$ even when only $x = 0$ is reachable.

a) Mechanized scope.: The Rocq artifact proves parameterized finite-tree acceptance, trace erasure and lifting, exact local framing, and preservation under licensed normalization. It also formalizes entry polarity and scalar call/alias properties. The rules in this section compose these interfaces at paper level. Correct realization of transfer, resolution, the observation schedule, and solver adapters is a premise of the implementation guarantee.

V. COMPARATIVE CASE STUDIES

We select cases that expose distinct requirements of a specification test oracle: arithmetic safety, quantified functional relations, and spatial ownership. Each case identifies its source, input, observed tool outcome, and control. These are recorded case studies rather than aggregate results for the full benchmark. The evidence inventory pins the original sources, logs, and proof receipts. The read cases use CN/Fulminate revision `39efc873f` (2026-04-16); least-root and divisibility use `181d04773` (2026-10-07). Both archived version strings carry a `dirty` suffix, and binary hashes identify the tested builds. These historical results remain separate from the latest-version evaluation specified in Section VI.

TABLE II
NEGATION MUTANT: CONTROLS AND THE BOUNDARY DISCREPANCY. RUNTIME REJECTION DENOTES FULMINATE EXIT CODE 2.

Initial value	Fulminate	TESPEC
`INT_MIN`	Accept (exit 0)	FAIL (safety)
0	Accept (exit 0)	PASS
$-1, 1, 7,$ `INT_MAX`	Runtime rejection	FAIL

A. An integer-boundary violation missed at runtime

The primary `read` program returns the value in an owned cell and preserves that cell. Listing 3 changes only the return expression. The CN and QCP versions express the same ownership and value requirements.

```
int read(int *p)
/*@ With old
    Require data_at(p, int, old)
    Ensure __return == old &&
           data_at(p, int, old) */
{ return -*p; }
```

Listing 3. Negation mutant of read. The contract still requires the original value.

For a 32-bit signed integer, the input $old = -2^{31}$ makes the mathematical negation 2^{31} unrepresentable. C classifies an expression result outside its type's representable range as undefined behavior (Section 6.5, paragraph 5 of the C11 draft [13]). The entry cell is valid; it is the executed negation that violates safety.

The recorded Fulminate pipeline uses `cn instrument --run` with GCC. Both the pipeline and a subsequent invocation of the generated executable exit with code 0; the latter produces no diagnostic. TESPEC reports a satisfiable entry and an invalid safety obligation, yielding FAIL. For this input the range check specializes to the false proposition

$$-2^{31} \leq 2^{31} \leq 2^{31} - 1.$$

The decisive evidence is the arithmetic-safety check, rather than a guessed native return value. The archived log does not record that value.

The controls in Table II distinguish the missed safety violation from ordinary incorrect return values. The correct read implementation also passes at `INT_MIN`. This is a missed violation in the recorded runtime configuration. Compiler and sanitizer configuration form part of that observation; a sanitizer-augmented baseline is a separate configuration in the evaluation protocol. The TESPEC result here is automatic solver evidence, not an independently replayed Rocq certificate.

B. A quantified minimum without a finite monitor

For the `root_cell` example, $n = 13$ produces the value 4, while Equation (1) still quantifies over all nonnegative integers. The recorded CN instrumenter rejects the original postcondition because it cannot infer an upper bound for the quantified variable. The same limitation appears in the archived Bennet and Darcy configurations. Thus this case exposes an assertion-support boundary, rather than an incorrect runtime answer.

The TESPEC postcondition-only replay initially returns UNKNOWN. Proof completion checks its complete group of 26 original obligations and produces PASS. The proof reasons directly about the quantified relation. As a control, the artifact proves its equivalence to the finite formula in Equation (2); that formula passes Fulminate on the same fixture. The comparison isolates the additional work required to obtain an executable monitor from a valid mathematical specification.

The intermediate assertion provides a second observation site. Replacing $\mathsf{LeastRoot}(n, r)$ by $\mathsf{LeastRoot}(n, r-1)$ preserves the program execution but changes the assertion to $\mathsf{LeastRoot}(13, 3)$. The archived 27-obligation group proves the other obligations and refutes this assertion VC. Its receipt does not admit a complete execution-counterexample certificate, so this is reported as a checked VC refutation. It demonstrates that internal specification changes can be observed even when inputs and outputs are unchanged.

C. Existence and nonexistence in one contract

The divisibility program returns whether `a % b == 0`, under $b > 0$. Its mathematical specification distinguishes two proof tasks:

$$(r = 1 \wedge \exists q \in \mathbb{Z}.\ a = qb) \ \vee \ (r = 0 \wedge \forall q \in \mathbb{Z}.\ a \neq qb).$$

The inputs $(6, 3)$ and $(7, 3)$ exercise both branches. The first proof supplies $q = 2$; the second excludes every integer witness. Both are positive conformance tests. Automatic processing leaves both unresolved, and proof completion checks each complete six-obligation execution group. The receipts record their logical dependencies and the kernel-checking route.

The finite remainder-based CN contract passes both inputs. Adding its logically redundant, unbounded universal consequence causes the tested instrumenter to reject the assertion with a lower-bound inference diagnostic. A separately kernel-checked Rocq lemma establishes the equivalence and the redundancy of that consequence. Together with least-root, this case shows that a formulation change can determine whether a runtime oracle is available, even when the intended property is unchanged.

D. A spatial mismatch resolved by either proof route

Two read variants omit the owned cell from their return contract. The body still retains that cell. In the exact resource interpretation used here, a nonempty owned heap does not satisfy `emp`. An automatically added frame for user heap would hide precisely this specification defect.

Across six integer inputs per variant, the recorded automatic run with spatial SMT disabled leaves 12 cases UNKNOWN. Independent proof-completion jobs refute the original spatial obligations and check entry feasibility. Enabling spatial SMT resolves those same 12 cases as FAIL, with countermodels checked against the finite-heap interpretation and agreement with the existing kernel-checked refutations. The remaining 49 raw classifications are unchanged, including all 22 passing

cases. These solver results retain their automatic-evidence label; agreement with a checked refutation does not make the solver output itself a kernel certificate.

Fulminate also detects the missing-ownership variants. This control therefore isolates the effect of TESPEC's proof backend: a residual spatial obligation can represent a real defect, and a stronger solver can decide it without changing execution or rejecting correct controls. The full read group contains 61 variant/input pairs across ten paired configurations. Fulminate matches 60 expected outcomes; TESPEC with per-execution ITP or with spatial SMT matches all 61 under the group's declared policy. Seven of these outcomes are expected memory-access errors for specifications that supply no read permission, and are kept separate from ordinary assertion failures.

VI. Evaluation Design

The broader evaluation will test whether the case-study mechanisms transfer across programs and predicate families. The protocol fixes program groups, variants, input cases, expected outcomes, and resource accounting before aggregating results. Case studies in Section V are the current evidence; the following comparisons are the study design.

a) Baseline versions and reproducibility.: For each new experimental batch, CN verification and Fulminate instrumentation will use the same build of the latest commit on the official CN default branch, frozen with its dependencies for the entire batch. On 2026-10-17, the upstream head was `b4a2f00923`; this identifies the next baseline, not a completed measurement. We will archive full revision and dependency identifiers, executable hashes, build settings, compiler and runtime-check flags, and exact commands. Historical measurements retain their original versions; each claimed case and its controls will be rerun before contributing to the new comparison.

b) Program groups and comparison units.: Each group has a primary program and several implementation or specification variants, with multiple inputs per variant. Existing source variants are retained. Expected behavior is reviewed independently of a tool verdict, including whether a case exercises defined execution, a functional mismatch, a spatial mismatch, or an unsafe operation. Only the primary program requires full verification. Every selected variant/input pair is a test unit. Mutants from one program are correlated observations, so aggregate analysis will report both group-level coverage and per-input outcomes.

c) Comparators and research questions.: Fulminate [4] is the direct runtime-checking baseline on matched CN contracts and inputs. On the shared latest-version build, we will compare the standard runtime-check configuration with a sanitizer-augmented configuration for arithmetic and memory-safety cases. Original quantified contracts and proved executable reformulations are reported separately. COINS [14] is compared on the shared pure input/output relation under the same model and proof budget. A proved admission is positive evidence; an unproved rejection remains an unconfirmed prediction. This comparison isolates what execution-derived state and internal observations add. Gillian-C [15], [16] supplies an independent symbolic-testing comparison on a reviewed common feature subset. Unsupported inputs remain visible in coverage figures.

CN and QCP verify each primary program's original contract once, without adding test-input equalities. This comparison asks how early instance feedback and its cost relate to full verification; their denominators are kept separate. Within TESPEC, the four combinations of spatial SMT on/off and ITP on/off measure automatic decision coverage and the contribution of proof completion. The protocol asks: which violations are detected, which original assertions can be checked, which obligations require proofs, and what each completed decision costs?

d) Outcomes and cost accounting.: Reports retain pass, confirmed violation, unresolved, expected semantic execution error, unsupported input, and infrastructure error. Following the distinction between noncompliance and prover incapacity in proof-failure diagnosis [17], a proof timeout never counts as detection. For proof-producing routes, the report additionally records goal coverage, kernel replay, and admitted assumptions.

Each group receives a result-and-cost table. CN records verification time and peak RSS; Fulminate records instrumentation/compilation and test execution separately. QCP and TESPEC separate VC generation/execution, agent proof filling, proof compilation with assumption audit, and independent kernel checking. One execution's complete residual VC set is one ITP job, including retries. Agent comparisons use `gpt-6.1-sol` with fixed budgets and record input, output, and total tokens. Peak RSS is measured per stage with its process-scope definition; memory peaks are not summed. Repeated isolated runs will report average per-execution cost, retaining raw records and failed attempts. Reused proofs are identified so replay cost is not mistaken for fresh proof-generation cost.

VII. Discussion

a) Choosing a test oracle.: Runtime checking provides immediate feedback when a contract has an executable interpretation. Proof-based testing extends that workflow to original mathematical assertions and reuses the specification's lemmas. Its cost therefore depends on both the execution and the proof route. The spatial read cases show how improving automatic discharge can remove proof work while preserving the execution and its obligations. Quantified contracts illustrate where proof completion remains useful.

b) Validity of the comparison.: The selected cases explain mechanisms and do not estimate their prevalence across projects. Contract pairs must agree on integer widths, ownership, aliasing, and logical definitions. The recorded read discrepancy concerns 32-bit signed negation; no pointer-layout conversion is involved. General C comparisons additionally need matched target layouts and compiler settings. Candidate preconditions also affect which inputs are admitted, so expected behavior and input selection must be fixed independently of those candidates.

c) Scope of the guarantees.: The Rocq development establishes reference observation semantics and the stated

execution-tree interfaces. The implementation additionally relies on C-to-VC correspondence, complete successor and boundary generation, and the selected memory interpretation. Automatic results trust the solver encoding; proof certificates check their exported propositions and admitted assumptions. These dependencies are recorded with the evidence rather than inferred from a single success label. Finite instance acceptance covers its selected domain; full verification covers the original precondition.

VIII. Related Work

a) Executable specification oracles.: The oracle problem separates generating inputs from deciding whether their behaviors are correct [5]. E-ACSL instruments C programs to check an executable subset of ACSL annotations [18]. SLICK checks separation-logic specifications at runtime, addressing implicit footprints and existential witnesses [19]. For concrete stack–heap states, Brotherston et al. study model checking of symbolic-heap assertions with user-defined inductive predicates [20]. CN expresses ownership and functional requirements in separation-logic refinement types [2]; Fulminate compiles its specifications into C checks that track ownership transfer [4]. TESPEC checks original assertions as logical obligations over reached symbolic states. Fulminate is the direct runtime comparator because it checks matched ownership contracts on the same C examples.

b) Testing with logical specifications.: QuickCheck evaluates executable properties on generated inputs [21]. QuickChick connects property-based testing in Rocq to logical specifications [22], [23]; FuzzChick adds coverage-guided, type-aware mutations [24]. HOL-TestGen derives test theorems and concrete data in Isabelle/HOL, retaining the hypotheses needed to generalize successful tests [25]. COINS seeks Rocq proofs that supplied input/output examples are admitted by relational specifications [14]. Its negative-case evaluation searches for unwanted acceptance; absence of that proof does not establish a checked rejection. TESPEC obtains the behavior by executing C and checks safety, internal observations, and postconditions. Comparing a direct instance proof oracle on the shared pure fragment isolates the value of these execution-derived observations.

c) Diagnosing failed verification.: StaDy combines specification instrumentation and dynamic symbolic execution to distinguish program/specification noncompliance, weak subcontracts, and prover incapacity [17]. Confirming prover incapacity requires complete exploration of the original input domain; incomplete exploration remains inconclusive. This distinction is central to interpreting a failed proof: it need not identify a program defect. StaDy seeks counterexamples through executable checks and controlled subcontract substitution. TESPEC retains the original assertion as a proposition at an executed state, allowing either a proof or a refutation to complete the oracle. Its verdict rules additionally require a reachable violating state before an abstract refutation becomes a test failure.

d) Symbolic execution and input generation.: Symbolic execution characterizes paths by constraints [26]. DART and CUTE combine concrete execution with symbolic constraints to generate new tests [27], [28]; KLEE scales symbolic test generation to systems code [29]. For heap-manipulating programs, CSF combines separation-logic-based generation of valid inputs with concolic execution [30]. Bennet generates heaps satisfying CN preconditions [31], and Darcy combines randomized choices with SMT constraints in a code–specify–test–debug–prove workflow [32]. These methods improve the inputs and paths available to an oracle. TESPEC focuses on deciding the original logical assertions on a selected domain. Gillian offers a memory-model-parametric symbolic execution framework and C instances for symbolic testing and separation-logic verification [15], [16]. Its independent execution infrastructure makes Gillian-C a comparator for the observation and proof workflow.

e) Verification and semantic interfaces.: VeriFast checks C and Java programs with separation-logic contracts [3]; Dafny integrates specifications and automatic functional verification [33]. CN and QCP [7] supply the full-verification comparison on our paired contracts: each primary program is checked against its original, uninstantiated contract. Exact separation logic separates coverage of behavior from the reachability of reported behaviors [12]. Verification-preserving inlining identifies conditions under which expanding calls and loops preserves verification in separation-logic tools [6]. The Verified Software Toolchain [34] and foundations for translational separation-logic verifiers [35] make semantic interfaces explicit across verification components. Our reference interfaces specialize this concern to copied observations: execution coverage, faithful assertion resolution, and oracle evidence must describe the same selected states.

f) Obligation solving and proof completion.: SMT-based separation-logic procedures discharge substantial supported fragments [8], [9]. Spatial SMT serves this role in TESPEC, with residual obligations passed to proof completion. CoqHammer reconstructs proofs suggested by external provers [10]; SMTCoq checks solver certificates with a verified Coq checker [11]. Tactician and Proverbot9001 guide tactic-level search [36], [37]. CoqPilot combines proof candidates with checking and repair [38], while Rango retrieves relevant proofs and premises as the proof state evolves [39]. These systems improve how a stated goal is discharged. TESPEC determines which goal checks an observed behavior and connects its checked polarity to an execution verdict. Proof-search improvements can therefore be used without redefining the instance or its acceptance criterion.

IX. Conclusion

TESPEC connects execution-based feedback with proof-oriented C contracts. Input bindings select the tested domain, actual program transitions determine the observed states, and proofs establish their specified properties. Copied observations, joint normalization, and exact local frames preserve this connection through the verification infrastructure. The case

studies show why a useful oracle must account for arithmetic safety, original quantified relations, and spatial ownership. The resulting workflow supports testing with the same specifications and logical libraries used for full verification.

Data Availability

The accompanying artifact includes the implementation identities, original case-study sources and logs, proof certificates, and the Rocq reference semantics. The paper's case-study inventory pins evidence by file hash and separates recorded observations from the broader evaluation protocol. The build script regenerates the manuscript and its three vector method figures.

References

[1] J. C. Reynolds, "Separation logic: A logic for shared mutable data structures," in *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science*, 2002, pp. 55–74.

[2] C. Pulte, D. C. Makwana, T. Sewell, K. Memarian, P. Sewell, and N. Krishnaswami, "CN: Verifying systems C code with separation-logic refinement types," *Proceedings of the ACM on Programming Languages*, vol. 7, no. POPL, 2023.

[3] B. Jacobs, J. Smans, P. Philippaerts, F. Vogels, W. Penninckx, and F. Piessens, "VeriFast: A powerful, sound, predictable, fast verifier for C and Java," in *NASA Formal Methods*, ser. Lecture Notes in Computer Science, vol. 6617. Springer, 2011, pp. 41–55. [Online]. Available: https://people.cs.kuleuven.be/~bart.jacobs/nfm2011.pdf

[4] R. Banerjee, K. Memarian, D. Makwana, C. Pulte, N. Krishnaswami, and P. Sewell, "Fulminate: Testing CN separation-logic specifications in C," *Proceedings of the ACM on Programming Languages*, vol. 9, no. POPL, pp. 1260–1292, 2025.

[5] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015. [Online]. Available: https://philmcminn.com/publications/barr2015.pdf

[6] T. Dardinier, G. Parthasarathy, and P. Müller, "Verification-preserving inlining in automatic separation logic verifiers," *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, 2023. [Online]. Available: https://gauravpartha.ch/papers/inlining_oopsla23.pdf

[7] X. Wu, Y. Feng, X. Lu, T. Lin, K. Liu, Z. Wang, S. Wu, L. Xie, C. Yang, H. Zhong, Z. Zhang, J. Li, N. Zhan, Z. Hu, and Q. Cao, "QCP: A practical separation logic-based C program verification tool," *arXiv preprint arXiv:2505.12878*, 2026. [Online]. Available: https://arxiv.org/abs/2505.12878

[8] R. Piskac, T. Wies, and D. Zufferey, "Automating separation logic using SMT," in *Computer Aided Verification*, ser. Lecture Notes in Computer Science, vol. 8044. Springer, 2013, pp. 773–789. [Online]. Available: https://cs.yale.edu/homes/piskac/papers/2013PiskacWiesZuffreySepLog.pdf

[9] A. Reynolds, R. Iosif, C. Serban, and T. King, "A decision procedure for separation logic in SMT," in *Automated Technology for Verification and Analysis*, ser. ATVA. Springer, 2016, pp. 244–261. [Online]. Available: https://cvc4.github.io/publications/2016/RIS%2B16.pdf

[10] Ł. Czajka and C. Kaliszyk, "Hammer for Coq: Automation for dependent type theory," *Journal of Automated Reasoning*, vol. 61, pp. 423–453, 2018. [Online]. Available: https://link.springer.com/article/10.1007/s10817-018-9458-4

[11] B. Ekici, A. Mebsout, C. Tinelli, C. Keller, G. Katz, A. Reynolds, and C. Barrett, "SMTCoq: A plug-in for integrating SMT solvers into Coq," in *Computer Aided Verification*, ser. Lecture Notes in Computer Science, vol. 10427. Springer, 2017, pp. 126–133. [Online]. Available: https://mebsout.github.io/papers/cav2017.pdf

[12] P. Maksimović, C. Cronjäger, A. Lööw, J. Sutherland, and P. Gardner, "Exact separation logic: Towards bridging the gap between verification and bug-finding," in *37th European Conference on Object-Oriented Programming*, ser. Leibniz International Proceedings in Informatics, vol. 263. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, pp. 19:1–19:27. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2023.19

[13] ISO/IEC JTC1/SC22/WG14, "Programming languages—c," ISO/IEC, Tech. Rep. N1570, 2011, committee draft, April 12. [Online]. Available: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

[14] F. Yang, X. Li, S. Wang, J. An, Z. Sun, S. Feng, W. Wang, W. Wang, N. Zhan, and F. Xu, "How powerful are LLMs in generating formal program specifications?" in *International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 145 221–145 249. [Online]. Available: https://proceedings.mlr.press/v306/yang26am.html

[15] J. F. Santos, P. Maksimović, S.-É. Ayoun, and P. Gardner, "Gillian, part I: A multi-language platform for symbolic execution," in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI. ACM, 2020, pp. 927–942. [Online]. Available: https://j3fsantos.github.io/papers/gillian.pdf

[16] P. Maksimović, S.-É. Ayoun, J. F. Santos, and P. Gardner, "Gillian, part II: Real-world verification for JavaScript and C," in *Computer Aided Verification*, ser. CAV. Springer, 2021. [Online]. Available: https://vtss.doc.ic.ac.uk/publications/Maksimovic2021Gillian.html

[17] G. Petiot, N. Kosmatov, B. Botella, A. Giorgetti, and J. Julliand, "How testing helps to diagnose proof failures," *Formal Aspects of Computing*, vol. 30, no. 6, pp. 629–657, 2018. [Online]. Available: https://nikolai-kosmatov.eu/publications/petiot_kbgj_faoc_2018.pdf

[18] J. Signoles, N. Kosmatov, and K. Vorobyov, "E-ACSL, a runtime verification tool for safety and security of C programs (tool paper)," in *RV-CuBES 2017. An International Workshop on Competitions, Usability, Benchmarks, Evaluation, and Standardisation for Runtime Verification Tools*, ser. Kalpa Publications in Computing, G. Reger and K. Havelund, Eds., vol. 3. EasyChair, 2017, pp. 164–173. [Online]. Available: https://easychair.org/publications/paper/t6tV

[19] H. H. Nguyen, V. Kuncak, and W.-N. Chin, "Runtime checking for separation logic," in *Verification, Model Checking, and Abstract Interpretation*, 2008, pp. 203–217.

[20] J. Brotherston, N. Gorogiannis, M. Kanovich, and R. Rowe, "Model checking for symbolic-heap separation logic with inductive predicates," in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2016, pp. 84–96.

[21] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 2000, pp. 268–279.

[22] Z. Paraskevopoulou, C. Hritcu, M. Dénès, L. Lampropoulos, and B. C. Pierce, "Foundational property-based testing," in *Interactive Theorem Proving*, 2015, pp. 325–343.

[23] Z. Paraskevopoulou, A. Eline, and L. Lampropoulos, "Computing correctly with inductive relations," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2022. [Online]. Available: https://lemonidas.github.io/pdf/ComputingCorrectly.pdf

[24] L. Lampropoulos, M. Hicks, and B. C. Pierce, "Coverage guided, property based testing," *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, 2019. [Online]. Available: https://mhicks.me/papers/fuzzchick.pdf

[25] A. D. Brucker and B. Wolff, "HOL-TestGen: An interactive test-case generation framework," in *Fundamental Approaches to Software Engineering*, ser. Lecture Notes in Computer Science, vol. 5503. Springer, 2009, pp. 417–420. [Online]. Available: https://logicalhacking.com/publications/brucker.ea-hol-testgen-2009/

[26] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, 1976. [Online]. Available: https://research.ibm.com/publications/symbolic-execution-and-program-testing

[27] P. Godefroid, N. Klarlund, and K. Sen, "DART: Directed automated random testing," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2005, pp. 213–223. [Online]. Available: https://people.eecs.berkeley.edu/~ksen/papers/dart.pdf

[28] K. Sen, D. Marinov, and G. Agha, "CUTE: A concolic unit testing engine for C," in *Proceedings of the 10th European Software Engineering Conference and 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, 2005, pp. 263–272. [Online]. Available: https://osl.cs.illinois.edu/media/papers/sen-2005-sigsoft-cute.pdf

[29] C. Cadar, D. Dunbar, and D. Engler, "KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *Proceedings of the 8th USENIX Symposium on Operating Systems*

Design and Implementation. USENIX Association, 2008, pp. 209–224. [Online]. Available: https://www.usenix.org/legacy/events/osdi08/tech/full_papers/cadar/cadar.pdf

[30] L. H. Pham, Q. L. Le, Q.-S. Phan, and J. Sun, "Concolic testing heap-manipulating programs," in *Formal Methods – The Next 30 Years*, 2019, pp. 442–461.

[31] Z. K. Aamer and B. C. Pierce, "Bennet: Randomized specification testing for heap-manipulating programs," *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 3924–3953, 2025.

[32] Z. K. Aamer, R. Banerjee, H. Katsura, D. Kaloper-Meršinjak, D. J. Economou, K. Memarian, D. Makwana, N. Krishnaswami, B. C. Pierce, C. Pulte, and P. Sewell, "Code-specify-test-debug-prove: Flexibly integrating separation logic specification into conventional workflows," *Proceedings of the ACM on Programming Languages*, vol. 10, no. PLDI, 2026.

[33] K. R. M. Leino, "Dafny: An automatic program verifier for functional correctness," in *Logic for Programming, Artificial Intelligence, and Reasoning*, ser. Lecture Notes in Computer Science, vol. 6355. Springer, 2010, pp. 348–370. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-17511-4_20

[34] A. W. Appel, "Verified software toolchain," in *Programming Languages and Systems*, 2011, pp. 1–17.

[35] T. Dardinier, M. Sammler, G. Parthasarathy, A. J. Summers, and P. Müller, "Formal foundations for translational separation logic verifiers," *Proceedings of the ACM on Programming Languages*, vol. 9, no. POPL, 2025. [Online]. Available: https://gauravpartha.ch/papers/translational_foundations_popl25.pdf

[36] L. Blaauwbroek, J. Urban, and H. Geuvers, "The Tactician: A seamless, interactive tactic learner and prover for Coq," in *Intelligent Computer Mathematics*, ser. Lecture Notes in Computer Science, vol. 12236. Springer, 2020, pp. 271–277. [Online]. Available: https://arxiv.org/abs/2008.00120

[37] A. Sanchez-Stern, Y. Alhessi, L. K. Saul, and S. Lerner, "Generating correctness proofs with neural networks," in *Proceedings of the 4th ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. ACM, 2020, pp. 1–10. [Online]. Available: https://proverbot9001.ucsd.edu/papers/proverbot9001.pdf

[38] A. Kozyrev, G. Solovev, N. Khramov, and A. Podkopaev, "CoqPilot, a plugin for LLM-based generation of proofs," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2024, pp. 2382–2385. [Online]. Available: https://arxiv.org/abs/2410.19605

[39] K. Thompson, N. Saavedra, P. Carrott, K. Fisher, A. Sanchez-Stern, Y. Brun, J. F. Ferreira, S. Lerner, and E. First, "Rango: Adaptive retrieval-augmented proving for automated software verification," in *Proceedings of the 47th International Conference on Software Engineering*, 2025. [Online]. Available: https://www.cs.cornell.edu/~lerner/papers/rango-final.pdf